SENATI

Escuela de Tecnologías de la Información

GUÍA DE LABORATORIO

Guía 13

Introducción a la Inteligencia Artificial y al Aprendizaje Automático

Explica el uso de la IA

Escuela	Tecnologías de la Información
Carrera	Ingeniería de Software con Inteligencia Artificial
Módulo formativo	Artificial Intelligence with Machine Learning in Java (Oracle Academy)
Semestre	IV
Semana	13
Tarea de aprendizaje	Explica el uso de la IA
Herramientas	Java 21 (JDK) · Visual Studio Code · Extension Pack for Java
Instructor	Rembrandt Ubalde Enríquez
Estudiante	
Fecha	

Contenido

Objetivos de la guía 3
Materiales 3
Parte 1. Preparación del entorno en Visual Studio Code 4
1.1 Verificar o instalar el JDK 4
1.2 Instalar la extensión de Java en VS Code 4
1.3 Opción A: abrir el proyecto entregado 4
1.4 Opción B: crear el proyecto desde cero 4
1.5 Ejecutar los programas 5
Parte 2. Conceptos clave 6
2.1 Resumen del curso 6
2.2 Introducción a la Inteligencia Artificial 6
2.3 Datos e información 6
2.4 Categorización de datos 7
2.5 Aprendizaje automático 7
2.6 Aprendizaje supervisado y no supervisado 8
2.7 Clasificación y regresión 8
Parte 3. Prácticas en Visual Studio Code 10
Práctica 01. De datos a información 10
Práctica 02. Categorización de datos 11
Práctica 03. Flujo de trabajo del aprendizaje automático (regresión) 13
Práctica 04. Clasificación supervisada con k vecinos (KNN) 15
Práctica 05. Aprendizaje no supervisado: agrupar clientes 17
Práctica 06. Proyecto: Tutor IA 18
Parte 4. Errores frecuentes 22
Parte 5. Ejercicios propuestos 22
Preguntas de reflexión 22
Lista de cotejo 23

(Si el índice aparece vacío, haz clic derecho sobre él y elige «Actualizar campos».)

Objetivos de la guía

Al terminar esta guía serás capaz de:

- **Explicar** qué es la inteligencia artificial y cómo se usa hoy.
- **Diferenciar** datos de información y **categorizar** los datos según su tipo.
- **Describir** el flujo de trabajo del aprendizaje automático y **utilizarlo** en un programa Java.
- **Diferenciar** el aprendizaje supervisado del no supervisado, y la clasificación de la regresión.

Operación	Dónde se trabaja en esta guía
Utiliza aprendizaje automático	Parte 2 (sección 2.5), Prácticas 03 y 06
Diferencia entre aprendizaje supervisado y no supervisado	Parte 2 (sección 2.6), Prácticas 04 y 05
Diferencia entre datos e información	Parte 2 (secciones 2.3 y 2.4), Prácticas 01 y 02
Diferencia entre clasificación y regresión	Parte 2 (sección 2.7), Prácticas 03, 04 y 06

Materiales

- Computadora con Windows 10/11, **JDK 17 o superior** (recomendado JDK 21) y Visual Studio Code con **Extension Pack for Java**.
- Proyecto **Semana13_IA_Java.zip** entregado por el instructor.

Parte 1. Preparación del entorno en Visual Studio Code

Esta guía trae el proyecto de VS Code **Semana13_IA_Java** con todas las clases listas. Puedes abrirlo directamente (opción A) o crearlo tú desde cero y escribir cada clase (opción B, recomendada para practicar).

1.1 Verificar o instalar el JDK

Abre una terminal (PowerShell) y escribe:

Terminal de VS Code

```
java -version
javac -version
```

▶ Resultado esperado (versión 17 o superior)

```
java version "21.0.x" ...
javac 21.0.x
```

Si aparece **«java no se reconoce como un comando»**, instala el **JDK 21** (Oracle JDK desde oracle.com/java o Eclipse Temurin desde adoptium.net). Durante la instalación marca las opciones **Add to PATH** y **Set JAVA_HOME**, y luego cierra y vuelve a abrir VS Code.

> ⚠ **Atención:** Los programas de esta guía usan `record`, `switch` con flechas y otras novedades de Java moderno. Necesitas **Java 17 o superior**; con Java 8 o 11 aparecerán errores de compilación.

1.2 Instalar la extensión de Java en VS Code

1. En VS Code abre la vista de **Extensiones** (Ctrl + Shift + X).
2. Busca **Extension Pack for Java** (de Microsoft) e instálala. Incluye el soporte del lenguaje, el depurador, el ejecutor de pruebas y el administrador de proyectos.
3. Reinicia VS Code si te lo solicita.

1.3 Opción A: abrir el proyecto entregado

1. Descomprime **Semana13_IA_Java.zip** en tu carpeta **Documentos**.
2. En VS Code entra a **Archivo > Abrir carpeta…** y selecciona la carpeta **Semana13_IA_Java** (la que contiene `src`, no la carpeta de arriba).
3. Elige **Sí, confío en los autores** si VS Code lo pregunta.
4. Espera a que en la barra inferior aparezca **Java: Ready** (la primera vez puede tardar un minuto).

1.4 Opción B: crear el proyecto desde cero

1. Pulsa **Ctrl + Shift + P**, escribe **Java: Create Java Project** y elige **No build tools**.
2. Selecciona **Documentos** como ubicación y escribe el nombre **Semana13_IA_Java**.
3. VS Code crea las carpetas `src`, `lib`, `bin` y el archivo `src/App.java` (puedes borrarlo).

4. Crea una carpeta **datos** junto a `src`. En la carpeta `src` crea cada clase con clic derecho > **Nuevo archivo**, usando el nombre exacto que indica la guía.

La estructura final del proyecto debe quedar así:

Estructura del proyecto

```
Semana13_IA_Java/
├── .vscode/settings.json
├── datos/
│   └── estudiantes.csv
└── src/
    ├── P01_DatosInformacion.java
    ├── P02_CategorizacionDatos.java
    ├── P03_FlujoML.java
    ├── P04_ClasificacionKNN.java
    ├── P05_NoSupervisado.java
    └── P06_ProyectoTutorIA.java
```

⚠ **Atención:** En Java el **nombre del archivo** debe ser idéntico al de la clase pública, respetando mayúsculas: la clase `public class P01_DatosInformacion` va en `P01_DatosInformacion.java`.

1.5 Ejecutar los programas

1. Abre el archivo (por ejemplo `P01_DatosInformacion.java`).
2. Haz clic en **Run** que aparece encima de `public static void main`, o en el botón ▶ de arriba a la derecha.
3. El resultado aparece en la **TERMINAL**. Si el programa pide datos con `Scanner`, escríbelos allí y presiona Enter.
4. Compara lo que ves con el recuadro verde **Resultado esperado**.

También puedes compilar y ejecutar desde la terminal, ubicado en la carpeta del proyecto:

Terminal de VS Code

```
javac -encoding UTF-8 -d bin src\*.java
java -cp bin P01_DatosInformacion
```

💡 **Consejo:** Si las tildes o la ñ se ven como símbolos raros (por ejemplo `INFORMACI?N`), escribe una vez `chcp 65001` en la terminal y vuelve a ejecutar.

💡 **Consejo:** Para depurar: haz clic a la izquierda del número de línea para poner un **punto de interrupción** (punto rojo) y elige **Debug** encima del `main`. Con **F10** avanzas línea por línea y con **F11** entras a un método; el panel **Variables** muestra los valores en cada momento.

Parte 2. Conceptos clave

2.1 Resumen del curso

El módulo **Artificial Intelligence with Machine Learning in Java** es un curso oficial de **Oracle Academy**. Su objetivo es que desarrolles algoritmos de IA programándolos tú mismo en Java, para entender cómo funcionan por dentro y no solo usarlos como una «caja negra». Las semanas finales del módulo siguen esta ruta:

Semana	Tema	Lo que construirás
13	Introducción a la IA y al aprendizaje automático	Programas que convierten datos en información, regresión, k vecinos y agrupamiento.
14 - 15	Árboles y recursividad	Un árbol binario, sus recorridos y el juego Sí/No que aprende.
16	Entropía y el algoritmo ID3	Un árbol de decisión que se construye solo a partir de los datos.
17	Evaluaciones semestrales	—

2.2 Introducción a la Inteligencia Artificial

La **inteligencia artificial (IA)** es la rama de la informática que crea sistemas capaces de realizar tareas que normalmente requieren inteligencia humana: reconocer imágenes, entender el lenguaje, tomar decisiones o aprender de la experiencia.

Concepto	Descripción	Ejemplo
Inteligencia artificial	Campo general: máquinas que muestran un comportamiento inteligente.	Un asistente virtual.
Aprendizaje automático (Machine Learning)	Parte de la IA donde el sistema **aprende de los datos** en lugar de seguir reglas escritas a mano.	Un filtro de spam que mejora con cada correo.
Aprendizaje profundo (Deep Learning)	Parte del ML que usa **redes neuronales** de muchas capas.	Reconocimiento facial.

Usos de la IA hoy: recomendaciones de Netflix o YouTube, detección de fraudes en tarjetas, diagnóstico médico por imágenes, traducción automática, chatbots, autos autónomos, control de calidad en fábricas y predicción de la demanda en tiendas.

2.3 Datos e información

Nivel	Qué es	Ejemplo (Práctica 01)
Datos	Hechos o valores **sin procesar** ni interpretar.	19.5, 21.0, 22.4, 18.9 …
Información	Datos **procesados y organizados** que tienen significado.	«La temperatura promedio fue 21.39 °C; el sábado fue el día más caluroso».
Conocimiento / decisión	Uso de la información para **actuar**.	«Aumentar el stock de helados».

Los algoritmos de IA reciben **datos** y producen **información** útil (una predicción, un grupo, una recomendación). Por eso la calidad de los datos determina la calidad del modelo: «basura entra, basura sale».

2.4 Categorización de datos

Tipo	Subtipo	Descripción	Ejemplo	En Java
Numérico	Discreto	Se **cuenta**; valores enteros.	Edad, n.º de hijos	`int`
Numérico	Continuo	Se **mide**; admite decimales.	Estatura, precio	`double`
Categórico	Nominal	Categorías **sin orden**.	Distrito, color	`String`
Categórico	Ordinal	Categorías **con orden**.	Básico < Intermedio < Avanzado	`enum`
Booleano	—	Solo dos valores.	¿Trabaja? Sí / No	`boolean`

Además, los datos pueden ser **estructurados** (tablas con filas y columnas, como un CSV o una base de datos) o **no estructurados** (texto libre, imágenes, audio, video). En un dataset, cada **fila** es un **ejemplo** o registro y cada **columna** es un **atributo** o característica.

2.5 Aprendizaje automático

Arthur Samuel (1959) lo definió como «el campo de estudio que da a las computadoras la capacidad de **aprender sin ser programadas explícitamente**».

Programación tradicional	Aprendizaje automático
El programador escribe las **reglas**.	El algoritmo **descubre las reglas** a partir de ejemplos.
Reglas + Datos → **Respuestas**	Datos + Respuestas → **Reglas (modelo)**
`if (horas > 5) nota = 15;`	`nota = 4.905 + 1.427 * horas` (aprendido en la Práctica 03)

¿Por qué ahora?

Las ideas del aprendizaje automático existen desde los años 50, pero su uso masivo es reciente porque se juntaron cuatro factores:

- **Datos masivos (Big Data):** internet, celulares y sensores generan enormes cantidades de datos para aprender.
- **Poder de cómputo:** procesadores gráficos (GPU) y la nube permiten entrenar modelos grandes en horas y no en meses.
- **Mejores algoritmos y librerías abiertas:** Java (Weka, Tribuo, DL4J), Python (scikit-learn, TensorFlow).
- **Inversión y demanda:** las empresas obtienen beneficios medibles y necesitan profesionales de IA.

Flujo de trabajo del aprendizaje automático

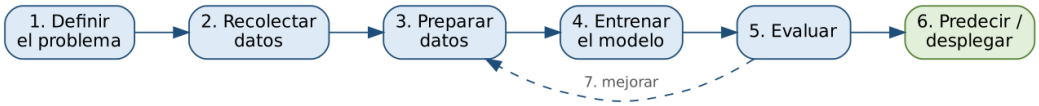


Figura 1. Flujo de trabajo de un proyecto de aprendizaje automático.

Paso	Qué se hace
1. Definir el problema	¿Qué queremos predecir o descubrir? ¿Es clasificación, regresión o agrupamiento?
2. Recolectar datos	Obtener ejemplos: archivos CSV, bases de datos, sensores, encuestas.
3. Preparar los datos	Limpiar errores, completar vacíos, escalar y **dividir** en entrenamiento y prueba.
4. Entrenar el modelo	El algoritmo ajusta sus parámetros con los datos de entrenamiento.
5. Evaluar	Medir el error con datos de **prueba** que el modelo no vio.
6. Predecir / desplegar	Usar el modelo con datos nuevos dentro de una aplicación.
7. Mejorar	Si el resultado no es bueno, volver a preparar datos o probar otro modelo.

2.6 Aprendizaje supervisado y no supervisado

Aspecto	Supervisado	No supervisado
Datos	**Con etiqueta**: cada ejemplo trae la respuesta correcta.	**Sin etiqueta**: solo las características.
Objetivo	Aprender a **predecir** la etiqueta de datos nuevos.	**Descubrir** estructuras o grupos ocultos.
Tareas	Clasificación y regresión.	Agrupamiento (clustering), reducción de dimensiones, detección de anomalías.
Algoritmos	Regresión lineal, k vecinos (KNN), **árboles de decisión (ID3)**.	K-Means, agrupamiento jerárquico.
Ejemplo	Predecir si un estudiante aprueba (Práctica 06).	Agrupar clientes por su comportamiento (Práctica 05).

> 💡 **Consejo:** Existe un tercer tipo: el **aprendizaje por refuerzo**, en el que un agente aprende por prueba y error recibiendo premios o castigos (por ejemplo, una IA que aprende a jugar ajedrez).

2.7 Clasificación y regresión

	Clasificación	Regresión
Predice	Una **categoría** (clase).	Un **número** continuo.
Pregunta	¿A qué grupo pertenece? ¿Sí o no?	¿Cuánto? ¿Qué valor tendrá?
Ejemplos	Manzana o naranja; aprueba o desaprueba; spam o no spam.	Nota de un examen; precio de una casa; ventas de mañana.
Cómo se evalúa	Exactitud: % de aciertos.	Error promedio (MAE), R^2.

	Clasificación	Regresión
En esta guía	Práctica 04 (KNN) y Práctica 06.	Práctica 03 y Práctica 06.

Parte 3. Prácticas en Visual Studio Code

Práctica 01. De datos a información

Una heladería registra la temperatura máxima de cada día. Esos números son **datos**; el programa los procesa para obtener **información** y tomar una **decisión**.

Archivo: src/P01_DatosInformacion.java

```
// P01_DatosInformacion.java - De DATOS a INFORMACIÓN
import java.util.Locale;

public class P01_DatosInformacion {
  public static void main(String[] args) {
    Locale.setDefault(Locale.US);                 // punto decimal en todas las PC

    // 1. DATOS: hechos sin procesar (temperatura máxima de una semana, en °C)
    String[] dias = {"Lun", "Mar", "Mié", "Jue", "Vie", "Sáb", "Dom"};
    double[] temp = {19.5, 21.0, 22.4, 18.9, 23.1, 24.6, 20.2};

    System.out.println("DATOS (sin procesar):");
    for (int i = 0; i < dias.length; i++) {
      System.out.printf("  %s = %.1f%n", dias[i], temp[i]);
    }

    // 2. PROCESAMIENTO: calculamos, comparamos y contamos
    double suma = 0;
    int iMax = 0, iMin = 0, calurosos = 0;
    for (int i = 0; i < temp.length; i++) {
      suma += temp[i];
      if (temp[i] > temp[iMax]) iMax = i;
      if (temp[i] < temp[iMin]) iMin = i;
      if (temp[i] > 22) calurosos++;
    }
    double promedio = suma / temp.length;

    // 3. INFORMACIÓN: datos procesados que tienen significado y sirven para decidir
    System.out.println("\nINFORMACIÓN:");
    System.out.printf("  Temperatura promedio de la semana: %.2f °C%n", promedio);
    System.out.printf("  Día más caluroso: %s (%.1f °C)%n", dias[iMax], temp[iMax]);
    System.out.printf("  Día más frío    : %s (%.1f °C)%n", dias[iMin], temp[iMin]);
    System.out.println("  Días con más de 22 °C: " + calurosos);

    // 4. CONOCIMIENTO / DECISIÓN
    String decision = (promedio > 21) ? "aumentar el stock de helados"
                    : "mantener el stock normal";
    System.out.println("\nDECISIÓN para la heladería: " + decision);
  }
}
```

▶ **Resultado esperado en la terminal**

```
DATOS (sin procesar):
  Lun = 19.5
  Mar = 21.0
  Mié = 22.4
  Jue = 18.9
  Vie = 23.1
  Sáb = 24.6
  Dom = 20.2

INFORMACIÓN:
  Temperatura promedio de la semana: 21.39 °C
  Día más caluroso: Sáb (24.6 °C)
  Día más frío    : Jue (18.9 °C)
  Días con más de 22 °C: 3

DECISIÓN para la heladería: aumentar el stock de helados
```

Análisis: los 7 números por sí solos no dicen mucho. Al procesarlos (sumar, comparar, contar) obtenemos información con significado: el promedio fue 21.39 °C y 3 días superaron los 22 °C. Con esa información el negocio decide.

> 💡 **Consejo:** `Locale.setDefault(Locale.US)` hace que todos los decimales se muestren con **punto**, sin importar la configuración regional de la PC.

Práctica 02. Categorización de datos

Representarás un dataset de estudiantes con un `record` de Java y aplicarás a cada atributo solo las operaciones que tienen sentido según su tipo.

Archivo: src/P02_CategorizacionDatos.java

```
// P02_CategorizacionDatos.java - Tipos de datos que usa la IA
import java.util.Locale;
import java.util.Map;
import java.util.TreeMap;

public class P02_CategorizacionDatos {

  // Dato CATEGÓRICO ORDINAL: las categorías tienen un orden
  enum NivelIngles { BASICO, INTERMEDIO, AVANZADO }

  // Cada registro (fila) describe a un estudiante con 6 atributos (columnas)
  record Estudiante(String nombre,        // identificador (no se usa para aprender)
          int edad,             // numérico DISCRETO (se cuenta)
          double estatura,      // numérico CONTINUO (se mide)
          String distrito,      // categórico NOMINAL (sin orden)
          NivelIngles ingles,   // categórico ORDINAL (con orden)
          boolean trabaja) { }  // BOOLEANO (sí / no)

  public static void main(String[] args) {
    Locale.setDefault(Locale.US);
    Estudiante[] datos = {
      new Estudiante("Ana",   19, 1.62, "Comas",       NivelIngles.INTERMEDIO, false),
      new Estudiante("Luis",  22, 1.75, "Los Olivos",  NivelIngles.BASICO,     true),
      new Estudiante("Rosa",  20, 1.58, "Comas",       NivelIngles.AVANZADO,   false),
      new Estudiante("Pedro", 25, 1.80, "Independencia", NivelIngles.BASICO,   true),
      new Estudiante("Carla", 21, 1.66, "Los Olivos",  NivelIngles.INTERMEDIO, true),
      new Estudiante("Jorge", 19, 1.70, "Comas",       NivelIngles.AVANZADO,   false),
    };

    System.out.println("Atributo   | Tipo de dato          | Operación válida");
    System.out.println("-----------+-----------------------+-----------------------");
    System.out.println("edad       | Numérico discreto     | promedio, máximo");
    System.out.println("estatura   | Numérico continuo     | promedio, rango");
    System.out.println("distrito   | Categórico nominal    | contar frecuencias");
    System.out.println("ingles     | Categórico ordinal    | comparar (mayor/menor)");
    System.out.println("trabaja    | Booleano              | contar sí / no");

    // Numéricos: se pueden promediar
    double sumaEdad = 0, sumaEst = 0;
    for (Estudiante e : datos) { sumaEdad += e.edad(); sumaEst += e.estatura(); }
    System.out.printf("%nEdad promedio    : %.2f años%n", sumaEdad / datos.length);
    System.out.printf("Estatura promedio: %.2f m%n", sumaEst / datos.length);

    // Nominal: NO se promedia, se cuenta cuántas veces aparece cada categoría
    Map<String, Integer> frecuencia = new TreeMap<>();
    for (Estudiante e : datos) frecuencia.merge(e.distrito(), 1, Integer::sum);
    System.out.println("Estudiantes por distrito: " + frecuencia);

    // Ordinal: sí se puede comparar usando su posición (ordinal)
    System.out.print("Inglés intermedio o superior: ");
    for (Estudiante e : datos) {
      if (e.ingles().ordinal() >= NivelIngles.INTERMEDIO.ordinal()) {
        System.out.print(e.nombre() + " ");
      }
    }
    // Booleano: se cuentan los verdaderos
    long trabajan = java.util.Arrays.stream(datos).filter(Estudiante::trabaja).count();
    System.out.println("\nEstudiantes que trabajan: " + trabajan + " de "
        + datos.length);
  }
}
```

▶ **Resultado esperado en la terminal**

```
Atributo   | Tipo de dato          | Operación válida
-----------+-----------------------+-----------------------
edad       | Numérico discreto     | promedio, máximo
estatura   | Numérico continuo     | promedio, rango
distrito   | Categórico nominal    | contar frecuencias
ingles     | Categórico ordinal    | comparar (mayor/menor)
trabaja    | Booleano              | contar sí / no

Edad promedio    : 21.00 años
Estatura promedio: 1.68 m
Estudiantes por distrito: {Comas=3, Independencia=1, Los Olivos=2}
Inglés intermedio o superior: Ana Rosa Carla Jorge
Estudiantes que trabajan: 3 de 6
```

Análisis: tiene sentido promediar la edad, pero **no** el distrito: en un dato nominal solo se cuentan frecuencias. El nivel de inglés es **ordinal**: con `enum` y `ordinal()` podemos preguntar si es «intermedio o superior». Elegir bien el tipo de cada dato es el primer paso para preparar datos para un algoritmo de IA.

Práctica 03. Flujo de trabajo del aprendizaje automático (regresión)

Recorrerás los pasos de la sección 2.5 para crear un modelo que **predice la nota** (un número) a partir de las horas de estudio. Es aprendizaje **supervisado** de **regresión**.

Archivo: src/P03_FlujoML.java

```
// P03_FlujoML.java - Flujo de trabajo del aprendizaje automático (REGRESIÓN)
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Locale;
import java.util.Random;

public class P03_FlujoML {
  public static void main(String[] args) {
    Locale.setDefault(Locale.US);
    Random azar = new Random(42);  // semilla: mismos datos en todas las PC

    // PASO 1. RECOLECTAR DATOS: horas de estudio (x) y nota obtenida (y)
    List<double[]> datos = new ArrayList<>();
    for (int i = 0; i < 30; i++) {
      double horas = Math.round((1 + 9 * azar.nextDouble()) * 10) / 10.0;
      double nota = 5 + 1.4 * horas + azar.nextGaussian() * 1.2;
      nota = Math.round(Math.min(20, Math.max(0, nota)) * 10) / 10.0;
      datos.add(new double[]{horas, nota});
    }
    System.out.println("PASO 1. Datos recolectados: " + datos.size() + " estudiantes");

    // PASO 2. PREPARAR: mezclar y dividir 80 % entrenamiento / 20 % prueba
    Collections.shuffle(datos, azar);
    int corte = (int) (datos.size() * 0.8);
    List<double[]> train = datos.subList(0, corte);
    List<double[]> test = datos.subList(corte, datos.size());
    System.out.println("PASO 2. Entrenamiento: " + train.size() + " | Prueba: "
        + test.size());

    // PASO 3. ENTRENAR: mínimos cuadrados  b1 = Σ(x-x̄)(y-ȳ) / Σ(x-x̄)²   b0 = ȳ -
    // b1·x̄
    double mx = 0, my = 0;
    for (double[] d : train) { mx += d[0]; my += d[1]; }
    mx /= train.size(); my /= train.size();
    double num = 0, den = 0;
    for (double[] d : train) {
      num += (d[0] - mx) * (d[1] - my);
      den += (d[0] - mx) * (d[0] - mx);
    }
    double b1 = num / den, b0 = my - b1 * mx;
    System.out.printf("PASO 3. Modelo aprendido: nota = %.3f + %.3f * horas%n", b0, b1);

    // PASO 4. EVALUAR con datos que el modelo NO vio
    double sumaErr = 0, ssRes = 0, ssTot = 0, myTest = 0;
    for (double[] d : test) myTest += d[1];
    myTest /= test.size();
    System.out.println("PASO 4. Evaluación con datos de prueba:");
    System.out.println("   horas  real  predicha");
    for (double[] d : test) {
      double pred = b0 + b1 * d[0];
      System.out.printf("   %5.1f %5.1f %9.2f%n", d[0], d[1], pred);
      sumaErr += Math.abs(d[1] - pred);
      ssRes += Math.pow(d[1] - pred, 2);
      ssTot += Math.pow(d[1] - myTest, 2);
    }
    System.out.printf("   MAE = %.3f puntos | R² = %.3f%n", sumaErr / test.size(),
        1 - ssRes / ssTot);

    // PASO 5. PREDECIR (usar el modelo con datos nuevos)
    double nuevas = 7.5;
    System.out.printf("PASO 5. Con %.1f horas de estudio la nota estimada es %.2f%n",
             nuevas, b0 + b1 * nuevas);
  }
}
```

▶ **Resultado esperado en la terminal**

```
PASO 1. Datos recolectados: 30 estudiantes
PASO 2. Entrenamiento: 24 | Prueba: 6
PASO 3. Modelo aprendido: nota = 4.905 + 1.427 * horas
PASO 4. Evaluación con datos de prueba:
   horas  real  predicha
     5.3  10.6     12.47
     6.7  11.2     14.47
     9.5  19.6     18.46
     5.1  11.4     12.18
     7.5  16.8     15.61
     7.3  16.6     15.32
   MAE = 1.587 puntos | R² = 0.732
PASO 5. Con 7.5 horas de estudio la nota estimada es 15.61
```

Análisis: el programa no tiene ninguna regla escrita sobre notas; **aprendió** la ecuación `nota = 4.905 + 1.427 * horas` a partir de 24 ejemplos. Con los 6 de prueba se equivoca en promedio **1.59 puntos** y explica el **73 %** de la variación ($R^2 = 0.732$). Gracias a `new Random(42)` todos obtendrán los mismos datos.

Práctica 04. Clasificación supervisada con k vecinos (KNN)

El algoritmo **k vecinos más cercanos** clasifica un dato nuevo mirando la clase de los k ejemplos más parecidos (los de menor distancia). Aquí clasifica frutas en **manzana** o **naranja** según su peso y textura.

Archivo: src/P04_ClasificacionKNN.java

```
// P04_ClasificacionKNN.java - Aprendizaje SUPERVISADO de CLASIFICACIÓN (k vecinos)
import java.util.Arrays;
import java.util.Comparator;
import java.util.Locale;

public class P04_ClasificacionKNN {

  // Datos ETIQUETADOS: peso (g), textura (1 = lisa ... 10 = rugosa) -> fruta
  static double[][] X = {
    {150, 2}, {170, 1}, {140, 3}, {160, 2}, {130, 2},      // manzanas
    {180, 8}, {200, 9}, {190, 7}, {210, 8}, {175, 9}       // naranjas
  };
  static String[] y = {"manzana", "manzana", "manzana", "manzana", "manzana",
            "naranja", "naranja", "naranja", "naranja", "naranja"};

  static double distancia(double[] a, double[] b) {
    // Distancia euclidiana; la textura se multiplica por 10 para que pese igual que el
    // peso
    double dPeso = a[0] - b[0];
    double dTextura = (a[1] - b[1]) * 10;
    return Math.sqrt(dPeso * dPeso + dTextura * dTextura);
  }

  static String clasificar(double[] nueva, int k) {
    Integer[] indices = new Integer[X.length];
    for (int i = 0; i < X.length; i++) indices[i] = i;
    Arrays.sort(indices, Comparator.comparingDouble(i -> distancia(nueva, X[i])));
    int votosManzana = 0;
    System.out.print("   vecinos más cercanos:");
    for (int j = 0; j < k; j++) {
      int i = indices[j];
      System.out.printf(" %s(%.1f)", y[i], distancia(nueva, X[i]));
      if (y[i].equals("manzana")) votosManzana++;
    }
    System.out.println();
    return (votosManzana > k / 2) ? "manzana" : "naranja";
  }

  public static void main(String[] args) {
    Locale.setDefault(Locale.US);
    double[][] nuevas = {{155, 2}, {195, 8}, {172, 5}};
    for (double[] f : nuevas) {
      System.out.printf("Fruta nueva: peso=%.0f g, textura=%.0f%n", f[0], f[1]);
      System.out.println("   => CLASE PREDICHA: " + clasificar(f, 3));
    }
  }
}
```

▶ Resultado esperado en la terminal

```
Fruta nueva: peso=155 g, textura=2
   vecinos más cercanos: manzana(5.0) manzana(5.0) manzana(18.0)
   => CLASE PREDICHA: manzana
Fruta nueva: peso=195 g, textura=8
   vecinos más cercanos: naranja(11.2) naranja(11.2) naranja(15.0)
   => CLASE PREDICHA: naranja
Fruta nueva: peso=172 g, textura=5
   vecinos más cercanos: naranja(26.9) naranja(31.0) manzana(32.3)
   => CLASE PREDICHA: naranja
```

Análisis: la salida es una **categoría**, por eso es **clasificación**. La tercera fruta (172 g, textura 5) está entre ambos grupos: sus 3 vecinos votan 2 a 1 y gana **naranja**. Observa que la textura se multiplica por 10 para que tenga el mismo peso que los gramos en la distancia.

Práctica 05. Aprendizaje no supervisado: agrupar clientes

Ahora los datos **no tienen etiqueta**: solo sabemos el gasto mensual y las visitas de 10 clientes. El algoritmo **K-Means** descubre por sí solo dos grupos.

Archivo: src/P05_NoSupervisado.java

```
// P05_NoSupervisado.java - Aprendizaje NO SUPERVISADO: agrupar clientes (K-Means)
import java.util.Locale;

public class P05_NoSupervisado {
  public static void main(String[] args) {
    Locale.setDefault(Locale.US);
    // Datos SIN etiqueta: gasto mensual (S/) y visitas al mes de 10 clientes
    double[][] clientes = {
      {120, 2}, {150, 3}, {100, 1}, {130, 2}, {140, 3},
      {480, 9}, {520, 10}, {450, 8}, {500, 12}, {470, 9}
    };
    int k = 2;
    // Centroides iniciales: el primer cliente y el último
    double[][] centro = {clientes[0].clone(), clientes[clientes.length - 1].clone()};
    int[] grupo = new int[clientes.length];

    for (int it = 1; it <= 10; it++) {
      // ASIGNACIÓN: cada cliente va al centroide más cercano
      for (int i = 0; i < clientes.length; i++) {
        double d0 = dist(clientes[i], centro[0]), d1 = dist(clientes[i], centro[1]);
        grupo[i] = (d0 <= d1) ? 0 : 1;
      }
      // ACTUALIZACIÓN: cada centroide pasa al promedio de su grupo
      double[][] nuevo = new double[k][2];
      int[] n = new int[k];
      for (int i = 0; i < clientes.length; i++) {
        nuevo[grupo[i]][0] += clientes[i][0];
        nuevo[grupo[i]][1] += clientes[i][1];
        n[grupo[i]]++;
      }
      boolean cambio = false;
      for (int g = 0; g < k; g++) {
        nuevo[g][0] /= n[g]; nuevo[g][1] /= n[g];
        if (dist(nuevo[g], centro[g]) > 1e-9) cambio = true;
        centro[g] = nuevo[g];
      }
      System.out.printf("Iteración %d: centro0=(%.1f, %.1f) centro1=(%.1f, %.1f)%n",
          it, centro[0][0], centro[0][1], centro[1][0], centro[1][1]);
      if (!cambio) {
        System.out.println("Los centroides no cambiaron: CONVERGIÓ.");
        break;
      }
    }
    System.out.println("\nCliente  gasto  visitas  grupo");
    for (int i = 0; i < clientes.length; i++) {
      System.out.printf("  C%-2d  %7.0f  %6.0f  %5d%n", i + 1, clientes[i][0],
          clientes[i][1], grupo[i]);
    }
    System.out.println("\nEl algoritmo formó los grupos SIN conocer etiquetas.");
    System.out.println("Nosotros los interpretamos: "
        + "grupo 0 = ocasionales, grupo 1 = frecuentes.");
  }

  static double dist(double[] a, double[] b) {
    return Math.sqrt(Math.pow(a[0] - b[0], 2) + Math.pow(a[1] - b[1], 2));
  }
}
```

▶ **Resultado esperado en la terminal**

```
Iteración 1: centro0=(128.0, 2.2) centro1=(484.0, 9.6)
Iteración 2: centro0=(128.0, 2.2) centro1=(484.0, 9.6)
Los centroides no cambiaron: CONVERGIÓ.

Cliente  gasto  visitas  grupo
  C1       120       2      0
  C2       150       3      0
  C3       100       1      0
  C4       130       2      0
  C5       140       3      0
  C6       480       9      1
  C7       520      10      1
  C8       450       8      1
  C9       500      12      1
  C10      470       9      1

El algoritmo formó los grupos SIN conocer etiquetas.
Nosotros los interpretamos: grupo 0 = ocasionales, grupo 1 = frecuentes.
```

Análisis: en ningún momento le dijimos al programa quién es cliente frecuente. Él agrupó a los clientes parecidos y nosotros **interpretamos** los grupos. Esa es la diferencia con la Práctica 04, donde cada fruta ya traía su etiqueta.

Práctica 06. Proyecto: Tutor IA

Integrarás todo lo aprendido en un programa que lee un archivo CSV con el historial de 20 estudiantes y, para un estudiante nuevo, aplica **regresión** (nota estimada) y **clasificación** (aprueba o desaprueba).

Paso 1. Crear el archivo de datos

Archivo: datos/estudiantes.csv

```
horas,asistencia,nota,resultado
1.5,60,6.8,DESAPRUEBA
2.0,70,8.1,DESAPRUEBA
2.5,55,8.0,DESAPRUEBA
3.0,80,9.9,DESAPRUEBA
3.5,65,9.6,DESAPRUEBA
4.0,85,11.6,APRUEBA
4.0,50,9.2,DESAPRUEBA
4.5,90,12.4,APRUEBA
5.0,75,12.1,APRUEBA
5.5,95,13.9,APRUEBA
6.0,60,12.0,APRUEBA
6.0,85,13.6,APRUEBA
6.5,90,14.6,APRUEBA
7.0,70,14.0,APRUEBA
7.5,95,16.1,APRUEBA
8.0,80,16.0,APRUEBA
8.5,100,17.5,APRUEBA
9.0,85,17.4,APRUEBA
2.0,95,9.4,DESAPRUEBA
3.0,100,10.9,APRUEBA
```

Paso 2. Crear la clase del proyecto

Archivo: src/P06_ProyectoTutorIA.java

```
// P06_ProyectoTutorIA.java - PROYECTO: Tutor IA (regresión + clasificación con datos
// CSV)
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;
import java.util.Locale;
import java.util.Scanner;

public class P06_ProyectoTutorIA {

  record Registro(double horas, double asistencia, double nota, String resultado) { }

  // Lee el archivo CSV y convierte cada línea en un Registro
  static List<Registro> cargar(String ruta) throws IOException {
    List<Registro> lista = new ArrayList<>();
    List<String> lineas = Files.readAllLines(Path.of(ruta));
    for (int i = 1; i < lineas.size(); i++) {  // i = 1: saltamos la cabecera
      String[] c = lineas.get(i).split(",");
      lista.add(new Registro(Double.parseDouble(c[0]), Double.parseDouble(c[1]),
                Double.parseDouble(c[2]), c[3].trim()));
    }
    return lista;
  }

  // REGRESIÓN: predice un NÚMERO (la nota) a partir de las horas
  static double[] entrenarRegresion(List<Registro> d) {
    double mx = 0, my = 0;
    for (Registro r : d) { mx += r.horas(); my += r.nota(); }
    mx /= d.size(); my /= d.size();
    double num = 0, den = 0;
    for (Registro r : d) {
      num += (r.horas() - mx) * (r.nota() - my);
      den += (r.horas() - mx) * (r.horas() - mx);
    }
    double b1 = num / den;
    return new double[]{my - b1 * mx, b1};
  }

  // CLASIFICACIÓN: predice una CATEGORÍA (APRUEBA / DESAPRUEBA) con k = 3 vecinos
  static String clasificar(List<Registro> d, double horas, double asistencia) {
    List<Registro> orden = new ArrayList<>(d);
    // Escalamos: horas de 0 a 10 y asistencia de 0 a 100 -> ambas de 0 a 1
    orden.sort((a, b) -> Double.compare(dist(a, horas, asistencia), dist(b, horas,
        asistencia)));
    int aprueba = 0;
    for (int i = 0; i < 3; i++) {
      if (orden.get(i).resultado().equals("APRUEBA")) aprueba++;
    }
    return aprueba >= 2 ? "APRUEBA" : "DESAPRUEBA";
  }

  static double dist(Registro r, double h, double a) {
    return Math.hypot((r.horas() - h) / 10.0, (r.asistencia() - a) / 100.0);
  }

  public static void main(String[] args) throws IOException {
    Locale.setDefault(Locale.US);
    List<Registro> datos = cargar("datos/estudiantes.csv");
    System.out.println("=== TUTOR IA - SENATI ===");
    System.out.println("Registros cargados: " + datos.size());

    double[] modelo = entrenarRegresion(datos);
    System.out.printf("Modelo de regresión: nota = %.2f + %.2f * horas%n", modelo[0],
        modelo[1]);
```

```
    Scanner sc = new Scanner(System.in);
    System.out.print("Horas de estudio por semana: ");
    double horas = sc.nextDouble();
    System.out.print("Porcentaje de asistencia   : ");
    double asistencia = sc.nextDouble();

    double nota = modelo[0] + modelo[1] * horas;
    String resultado = clasificar(datos, horas, asistencia);
    System.out.printf("%n[Regresión]     Nota estimada : %.1f%n", nota);
    System.out.println("[Clasificación] Predicción    : " + resultado);
    if (resultado.equals("DESAPRUEBA")) {
      System.out.println("Consejo: aumenta tus horas de estudio y tu asistencia.");
    } else {
      System.out.println("¡Vas por buen camino! Mantén tu ritmo.");
    }
    sc.close();
  }
}
```

Paso 3. Ejecutar y probar con dos estudiantes

▶ **Resultado esperado (escribiendo 6.5 y 88)**

```
=== TUTOR IA - SENATI ===
Registros cargados: 20
Modelo de regresión: nota = 5.56 + 1.33 * horas
Horas de estudio por semana: 6.5
Porcentaje de asistencia   : 88

[Regresión]     Nota estimada : 14.2
[Clasificación] Predicción    : APRUEBA
¡Vas por buen camino! Mantén tu ritmo.
```

▶ **Resultado esperado (escribiendo 2.5 y 65)**

```
=== TUTOR IA - SENATI ===
Registros cargados: 20
Modelo de regresión: nota = 5.56 + 1.33 * horas
Horas de estudio por semana: 2.5
Porcentaje de asistencia   : 65

[Regresión]     Nota estimada : 8.9
[Clasificación] Predicción    : DESAPRUEBA
Consejo: aumenta tus horas de estudio y tu asistencia.
```

⚠ **Atención:** El programa lee `datos/estudiantes.csv` con una **ruta relativa**, que se busca desde la carpeta abierta en VS Code. Si aparece `NoSuchFileException`, revisa que abriste la carpeta del proyecto y no la carpeta de arriba.

💡 **Consejo:** Escribe los decimales con **punto** (6.5). Como el programa usa `Locale.US`, si escribes 6,5 aparecerá `InputMismatchException`.

Parte 4. Errores frecuentes

Mensaje o síntoma	Causa	Solución
«java no se reconoce como un comando»	El JDK no está instalado o no está en el PATH.	Instala el JDK 21 marcando **Add to PATH** y reinicia VS Code.
No aparece **Run** encima del `main`	Falta la extensión o VS Code aún carga el proyecto.	Instala **Extension Pack for Java** y espera a ver **Java: Ready**.
`class X is public, should be declared in a file named X.java`	El nombre del archivo no coincide con la clase.	Renombra el archivo exactamente igual que la clase.
`NoSuchFileException: datos\estudiantes.csv`	Se abrió otra carpeta o falta el archivo.	Abre la carpeta **Semana13_IA_Java** y verifica que exista `datos/estudiantes.csv`.
`InputMismatchException`	Se escribió una letra o un decimal con coma.	Escribe números con punto: `6.5`.
`records are not supported in -source 8`	Versión antigua de Java.	Instala JDK 17 o superior y selecciónalo con **Java: Configure Java Runtime**.
Las tildes salen como `?`	La terminal no usa UTF-8.	Ejecuta `chcp 65001` en la terminal.

Parte 5. Ejercicios propuestos

1. Modifica la Práctica 01 para procesar las notas de 10 estudiantes: calcula el promedio, la nota máxima y mínima, cuántos aprobaron (nota ≥ 10.5) y el porcentaje de aprobados. Si menos del 60 % aprobó, muestra «Programar reforzamiento».
2. Crea un `record Producto` con los atributos código, precio, categoría, talla (S/M/L), enOferta y stock. Indica el tipo de dato de cada atributo y muestra: precio promedio, productos por categoría y productos con talla M o superior.
3. En la Práctica 03 cambia la división a 70 % / 30 % y la semilla a 7. ¿Cómo cambian la ecuación, el MAE y el R^2? Explica por qué.
4. En la Práctica 04 clasifica la fruta (172 g, textura 5) con k = 1 y con k = 5. ¿Cambia la respuesta? ¿Por qué conviene que k sea impar?
5. Completa la tabla indicando si cada problema es supervisado o no supervisado y si es clasificación, regresión o agrupamiento: a) predecir el precio de un departamento; b) detectar correos spam; c) formar grupos de clientes con gustos parecidos; d) predecir la temperatura de mañana; e) diagnosticar si una radiografía muestra neumonía; f) organizar noticias por temas sin categorías previas.

Preguntas de reflexión

1. ¿En qué se diferencia un programa tradicional de un programa que aprende? Usa la Práctica 03 como ejemplo.
2. ¿Por qué no tiene sentido calcular el promedio de un dato nominal como el distrito?
3. ¿Por qué el modelo se evalúa con datos de prueba que no usó para entrenar?

4. Menciona dos usos de la IA en tu comunidad o centro de trabajo e indica qué tipo de aprendizaje usarían.

Lista de cotejo

- ☐ Instalé el JDK y la extensión de Java; abrí y ejecuté el proyecto en VS Code.
- ☐ Explico qué es la IA, el aprendizaje automático y el aprendizaje profundo.
- ☐ Diferencio datos de información (Práctica 01).
- ☐ Categorizo los atributos de un dataset (Práctica 02).
- ☐ Describo y apliqué el flujo de trabajo del aprendizaje automático (Práctica 03).
- ☐ Diferencio aprendizaje supervisado (Práctica 04) y no supervisado (Práctica 05).
- ☐ Diferencio clasificación y regresión (Prácticas 03, 04 y 06).
- ☐ Ejecuté el proyecto Tutor IA leyendo datos de un CSV (Práctica 06).
- ☐ Resolví al menos tres ejercicios propuestos.